

LABORATORIO DI SISTEMI OPERATIVI

Concorrenza, C11 atomic support

fabio.rossi@unipg.it

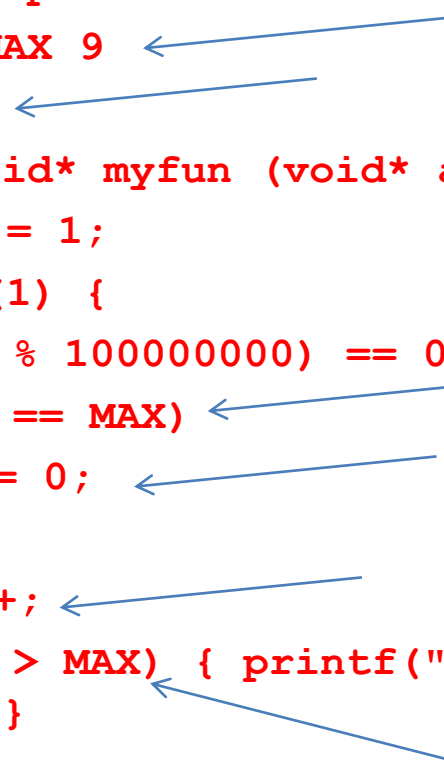
075 5855005

Race Condition

- è un problema di concorrenza
- dalle slide del Prof. Carpi: *"Si verifica quando più processi o threads leggono e scrivono dati in modo che il risultato finale dipende dall'ordine di esecuzione delle istruzioni dei processi"*
- **sezione critica**: porzione di codice eseguita da più threads/processi per la quale il risultato delle esecuzioni concorrenti dipende dalla sequenza con cui i threads/processi accedono a questa sezione di codice (e potrebbe non essere quello desiderato...)
- in questo caso si dice che la *sezione critica* contiene una *race condition*

(race_condition.c)

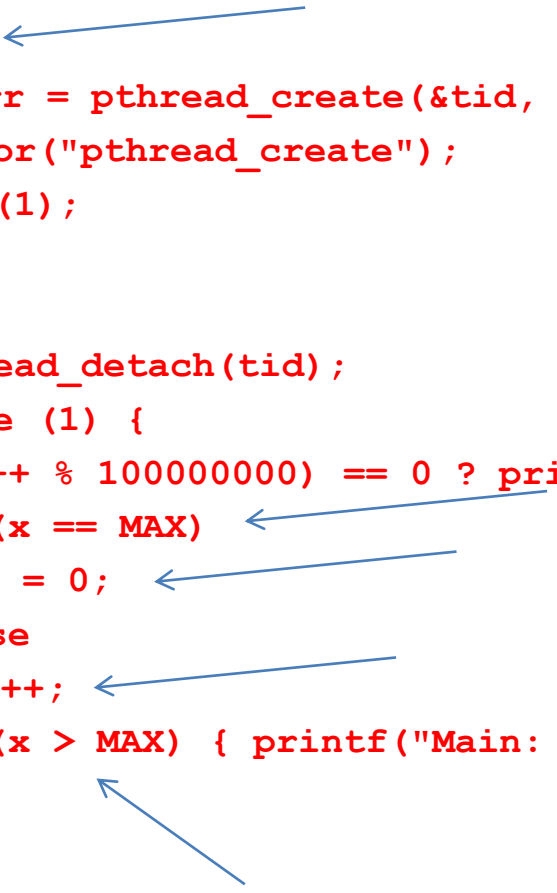
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#define MAX 9
int x;
static void* myfun (void* arg){
    long i = 1;
    while (1) {
        (i++ % 100000000) == 0 ? printf("Thread is going...\n") : i;
        if(x == MAX)
            x = 0;
        else
            x++;
        if(x > MAX) { printf("Thread: Sforato! x = %d\n", x);
        exit(1); }
    }
    pthread_exit((void *)0);
}
```



```
int main (void) {
    pthread_t tid; int err;
    long i = 1, status;

    x = 0;
    if ( err = pthread_create(&tid, NULL, &myfun, NULL) != 0 ) {
        perror("pthread_create");
        exit(1);
    }
    else {
        pthread_detach(tid);
        while (1) {
            (i++ % 100000000) == 0 ? printf("Main is going...\n") : i;
            if(x == MAX)
                x = 0;
            else
                x++;
            if(x > MAX) { printf("Main: Sforato! x = %d\n", x); exit(1); }
        }

        return 0;
    }
}
```



Race Condition

- la variabile globale **x** è oggetto di accesso ed incremento concorrenti da parte di entrambi i thread:

```
if (x == MAX)
    x = 0;
else
    x++;
```

- l'accesso (verifica del valore) e l'aggiornamento non sono indivisibili (non possono essere considerate un'unica operazione *atomica*)
- non sappiamo nulla di come i thread saranno schedulati

Race Condition

- risultato:

```
user1@debian10:~/SysCall/PThreads$ ./race_condition
Main going...
Thread going...
Thread: Sforato! x = 10
user1@debian10:~/SysCall/PThreads$
```

- la variabile **x** ha assunto un valore superiore a **MAX**

Race Condition

- risultato:

```
user1@debian10:~/SysCall/PThreads$ ./race_condition
Main going...
Thread going...
Thread: Sforato! x = 10
user1@debian10:~/SysCall/PThreads$
```

- è successo che uno dei thread ha letto il valore di **x** (es.: 8) e poi è stato descheduled; l'altro ha letto ed incrementato ($x=9$) ed è stato descheduled; il precedente, di nuovo schedulato, ha incrementato a sua volta ($x=10 > MAX$)!

Race Condition

```
if (x == MAX)
    x = 0;
else
    x++;
```

- è una *sezione critica*
- dobbiamo garantire che i thread abbiano accesso *mutuamente esclusivo* alle sezioni critiche

Mutua Esclusione

- Più soluzioni, tra cui:
 - semafori: ne parleremo più ampiamente in altra sede
 - *pthread_mutex*: parte della libreria *pthread*s. Ne parleremo brevemente...
 - supporto per le operazioni atomiche in C: *Atomic operations library*, a partire da C11
(<https://en.cppreference.com/w/c/atomic>)

Pthreads: Mutex

- da *man pthread_mutex_init(3)**: "A mutex is a MUTual EXclusion device, and is useful for protecting shared data structures from concurrent modifications, and implementing critical sections and monitors "
- in *pthread*s abbiamo a disposizione un "MUTual EXclusion device", le variabili **mutex**
- una *mutex* può avere due possibili stati: **locked** ("owned" da un thread) e **unlocked** (nessun thread è "owner")
- solo un thread alla volta può essere " owner "
- un thread che tenti di acquisire *ownership* su una *mutex* già *owned* da altro thread si blocca finché il thread attualmente *owner* non la rilascia

(*): per avere il *man* delle funzioni *pthread*s per le *mutex* ho dovuto installare il pacchetto **glibc-doc**

Pthreads: Fasi per una mutex

- si definisce una variabile ***pthread_mutex_t mutex*** per la singola sezione critica e la si inizializza
- si acquisisce l'accesso esclusivo alla sezione critica con ***pthread_mutex_lock(&mutex)***
- si elabora la sezione critica
- si rilascia l'accesso esclusivo con ***pthread_mutex_unlock(&mutex)***
- alla fine, quando la variabile mutex non serve più si può chiamare ***pthread_mutex_destroy*** che libera le risorse allocate per la mutex (deve essere unlocked)

C11 Atomic support

- un'alternativa in certi casi per gestire la concorrenza, introdotto in *C11*
- *C11* (più compiutamente *ISO/IEC 9899:2011*) è uno standard del passato per il linguaggio C
- tra le altre cose introduce il supporto per il *multi-threading*

C11 Atomic support

- introduce anche il supporto per variabili ed operazioni ***atomiche***
- *stdatomic.h*, qualificatore *Atomic* e varie primitive per le operazioni

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
//#include <stdatomic.h>

#ifdef NUM_THREADS
#define NUM_THREADS 4
#endif

/* _Atomic */ unsigned long counter = 0L;

enum {MAX_ITER = 100000000L};

void *incrementCounter(void *thr_id) {
    long tid;
    tid = (long)thr_id;
    printf("thread %ld started incrementing ID - %lu\n", tid, pthread_self());

    for (long i = 0L; i < MAX_ITER; ++i) {
        counter++;
    }

    pthread_exit(NULL);
}

```

(segue il main())

```

int main(int argc, char const *argv[]) {
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;

    for (t = 0; t < NUM_THREADS; t++) {
        rc = pthread_create(&threads[t], NULL, &incrementCounter, (void *)t);
        if (rc != 0) {
            printf("ERROR: pthread_create() call failed\n");
            exit(EXIT_FAILURE);
        }
    }

    for (t = 0; t < NUM_THREADS; t++) {
        pthread_join(threads[t], NULL);
    }
    printf("count = %ld\n", counter);

    exit(EXIT_SUCCESS);
}

```

- *l'intero esempio è liberamente adattato da [DelftStack](#)*

C11 Atomic support

- se eseguite più volte l'esempio precedente con un valore sufficientemente elevato di **MAX_ITER** potreste riscontrare un comportamento anomalo
- in particolare il valore finale della variabile **counter** non è quello atteso
- ciò perché i thread "competono" per la stessa variabile e **counter++** non è un'operazione atomica (lettura e incremento)
- se decommentate le parti in verde avrete il comportamento corretto

C11 Atomic support

- il qualificatore ***_Atomic*** permette di dichiarare variabili alle quali è garantito un accesso atomico
- per esempio, un'operazione di incremento, come **a++** che implica un accesso alla variabile (lettura del valore) ed un incremento, è garantita essere atomica
- occorre includere ***stdatomic.h***

C11 Atomic support

- in *stdatomic.h* sono definite delle typedef che creano "nuovi" tipi di dato analoghe alle seguenti:

Typedef name	Full type name
<i>atomic_char</i>	<i>_Atomic char</i>
<i>atomic_schar</i>	<i>_Atomic signed char</i>
<i>atomic_uchar</i>	<i>_Atomic unsigned char</i>
<i>atomic_short</i>	<i>_Atomic short</i>
<i>atomic_ushort</i>	<i>_Atomic unsigned short</i>
<i>atomic_int</i>	<i>_Atomic int</i>
<i>etc.</i>	

- **C11** introduce anche varie funzioni per compiere più operazioni su variabili in maniera atomica

C11 Atomic support

- ad esempio C11 introduce funzioni analoghe a:

`atomic_fetch_add( volatile A* obj, M arg )`

che atomicamente aggiunge `arg` al valore di `obj`

- per maggiori informazioni ved:

<https://en.cppreference.com/w/c/atomic>

Nell'esempio precedente, invece di utilizzare `_Atomic`, possiamo utilizzare `atomic_fetch_add()`:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <stdatomic.h>

#ifdef NUM_THREADS
#define NUM_THREADS 4
#endif

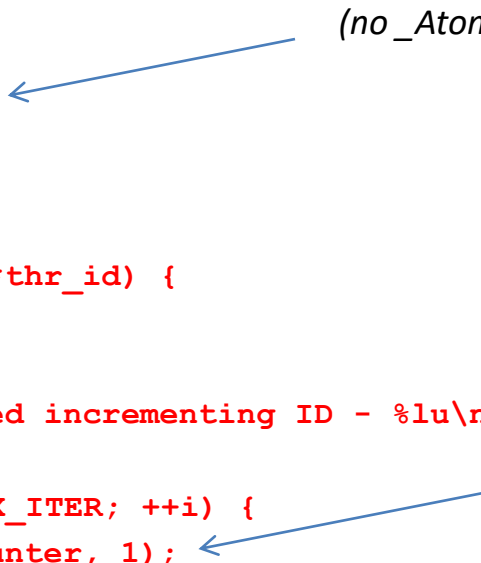
unsigned long counter = 0L;

enum {MAX_ITER = 100000000L};

void *incrementCounter(void *thr_id) {
    long tid;
    tid = (long)thr_id;
    printf("thread %ld started incrementing ID - %lu\n", tid, pthread_self());

    for (long i = 0L; i < MAX_ITER; ++i) {
        atomic_fetch_add(&counter, 1);
    }

    pthread_exit(NULL);
}
```



(no `_Atomic`)